Advanced Machine Learning

Non scholae, sed vitae discimus

Vincenzo Salvatore

Raffaele Mercurio

CORSO DI ADVANCED MACHINE LEARNING, UNIVERSITÀ LA SAPIENZA

Contents

1 Recap on Machine Learning and Computer Vision 5

1.1 Introduction to the Core Concepts 5

1.2 Image Classification 6

1.3 Parametric Models and Gradient-Based Optimization 6

1.3.1 Linear Regression ($f : \mathbb{R}^D \to \mathbb{R}$) 7

1.3.2 Logistic Regression ($f : \mathbb{R}^D \to [0,1]$) 7

1.3.3 Softmax Classifier ($f : \mathbb{R}^D \to \{0,\ldots,K\}$) 7

1.4 Advanced Optimization and Regularization 9

1.4.1 Regularization 9

1.4.2 Dropout 10

1.4.3 The Common Pattern of Modern Regularization 11

1.4.4 Stochastic Gradient Descent (SGD) 11

1.5 Backpropagation and Neural Networks 12

1.5.1 Backpropagation 12

1.5.2 Deeper Neural Networks 16

1.6 The Role of Activation Functions 16
1.6.1 1. ReLU (Rectified Linear Unit) 16
1.6.2 2. Leaky ReLU 16
1.6.3 3. ELU (Exponential Linear Units) 17
1.6.4 4. GELU (Gaussian Error Linear Unit) 17

1.7 Recap on Convolutional Neural Networks (CNNs) 18
1.7.1 Motivation: The Problem with Fully Connected Layers 18
1.7.2 Exploiting Image Structure: Locality and Stationarity 18
1.7.3 The Core Building Block: A Simple Convolutional Layer 19
1.7.4 Key Hyperparameters: Controlling the Convolution 19
1.7.5 Formal Mathematical Definition 20
1.7.6 Intuitive Examples: Classic Image Processing Filters 20
1.7.7 Output Shape and Parameter Counting for 3D Volumes 21
1.7.8 The Pooling Layer 22
1.7.9 Visualizing ConvNets 23

1. Recap on Machine Learning and Computer

1.1 Introduction to the Core Concepts

The contents of this chapter are largely based on the introductory slides of the course [1].

Key Word

- According to Arthur Samuel (1959), **Machine Learning** is the field of study that gives computers the ability to learn without being explicitly programmed.
- Tom Mitchell (1998) defined the well-posed learning problem: a computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

Machine learning is heavily applied in **Computer Vision**, which can be viewed from three main perspectives: as a science (exploring the computational model of human perception), as engineering (building systems that perceive the world), and through its applications.

Example

Some typical **Computer Vision Applications** include:

- **Medical imaging**: Supporting medical diagnosis and visualization.
- **Surveillance**: Tracking people in airports or train stations.

- **Autonomous driving**: Lane-keeping and pre-crash intervention.

1.2 Image Classification

A core task in computer vision is **Image Classification**: assuming a given set of discrete labels (e.g., {dog, cat, truck, plane}), the goal is to map an input digital image to one of these labels.

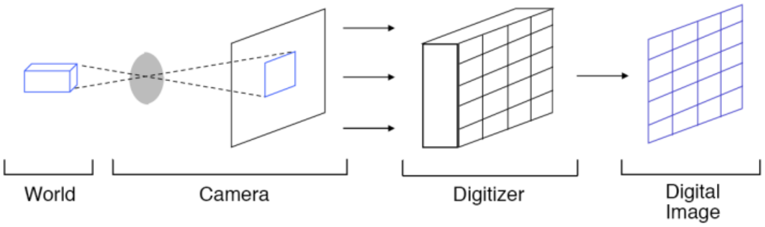


Figure 1.1: Illustration of the (pinhole) camera imaging process.

Note

The Imaging Process: An image is typically formed via a (pinhole) camera model and then passed through a digitizer to obtain a digital image, which the computer interprets as a large multi-dimensional array of numbers.

1.3 Parametric Models and Gradient-Based Optimization

The majority of machine learning models can be seen as **Parametric Models**. A parametric model takes an input x and uses a set of parameters (or weights) W and biases b to produce an output prediction z:

$$z = f(x, W) = Wx + b$$

While the exact nature of the prediction depends on the task, what unifies these models is the **Optimization Process**. We define a Loss function L that measures the error of our predictions, and we iteratively adjust the weights using **Gradient Descent**:

$$W^{t+1} = W^t - \alpha \nabla_W L$$

where α is the learning rate.

By simply changing the *activation function* applied to the linear score z and the corresponding *loss function*, we obtain different classical models: Linear Regression, Logistic Regression, and the Softmax Classifier.

1.3.1 Linear Regression ($f : \mathbb{R}^D \to \mathbb{R}$)

Linear Regression is used when the target variable is continuous. We use the identity activation function (i.e., no activation), so the predicted output $\hat{y}$ is exactly the linear score:

$$\hat{y} = z = Wx + b$$

We optimize this model by minimizing the **Mean Squared Error (MSE)** loss:

$$L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

1.3.2 Logistic Regression ($f : \mathbb{R}^D \to [0,1]$)

Logistic Regression is used for binary classification. We map the linear score z to a probability between 0 and 1 using the **Sigmoid** activation function $\sigma(z)$:

$$P(Y = 1|X = x) = \sigma(z) = \frac{1}{1+e^{-z}}$$

The appropriate loss function for this model is the **Binary Cross-Entropy Loss (BCE)**. For a dataset of N samples, it is defined as:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[\underbrace{y_i \log(\sigma(z_i))}_{\text{Active if } y=1} + \underbrace{(1-y_i)\log(1-\sigma(z_i))}_{\text{Active if } y=0}\right]$$

Note

Understanding the BCE Formula: This equation uses a clever mathematical trick to combine two mutually exclusive cases into a single line:

- When the true class is $y_i = 1$, the second term $(1-1) = 0$ disappears, and the loss evaluates only $-\log(\sigma(z_i))$, penalizing the model if it predicts a low probability.
- When the true class is $y_i = 0$, the first term 0 disappears, and the loss evaluates only $-\log(1-\sigma(z_i))$, penalizing the model if it predicts a high probability for class 1.

1.3.3 Softmax Classifier ($f : \mathbb{R}^D \to \{0,\dots,K\}$)

The Softmax Classifier generalizes Logistic Regression to multiple, mutually exclusive classes. It maps the raw scores z into a valid probability distribution over the classes using the **Softmax function**:

$$P(Y = k|X = x_i) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Note

Why Softmax? We want to interpret raw classifier scores as *probabilities*. The Softmax function transforms scores such that they are strictly positive and sum to 1.

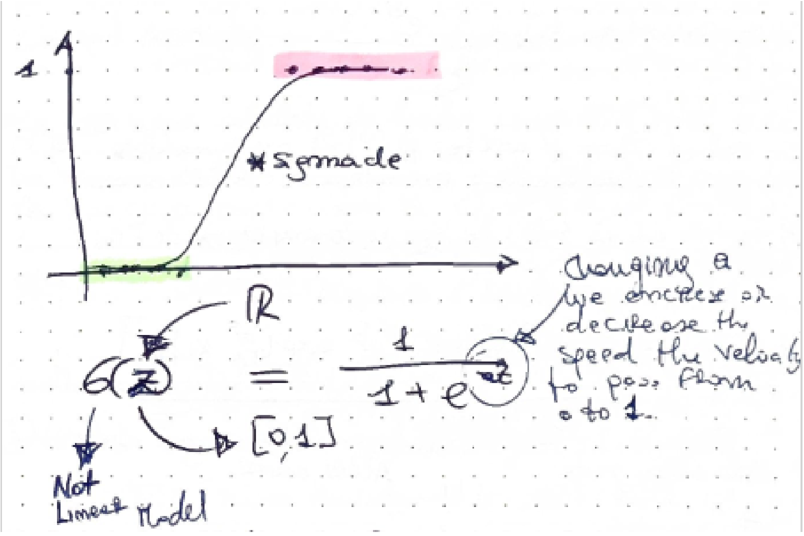

1
*Sigmoide
R
σ(z) = 1 / (1 + e^{-z})
[0,1]
Not Linear Model
changing z we increase or decrease the speed the velocity to pass from 0 to 1.

Figure 1.2: Illustration of the Sigmoid function in Logistic Regression.

Example

For an input image of size $32 \times 32 \times 3$ (e.g., CIFAR-10), the flattened array x has 3072 numbers. If we are classifying into 10 categories, the weight matrix W will have dimensions 10×3072, producing a 10×1 vector of raw class scores before the Softmax is applied.

We then optimize the model using the **Categorical Cross-Entropy Loss**, often referred to as the Negative Log-Likelihood (NLL). Over the entire dataset of N samples, expanding the Softmax probability formula, the loss becomes:

$$NLL(W,X) = -\sum_{i=1}^{N} \left[\underbrace{W_{y_i}^T x_i}_{\text{Punteggio della classe giusta}} - \underbrace{\log\left(\sum_{k=1}^{K} \exp(W_k^T x_i)\right)}_{\text{Somma dei punteggi delle altre classi}} \right]$$

Note

Understanding the Softmax NLL Formula: By looking at the expanded formula, we see that the optimizer tries to maximize the linear score $W_{y_i}^T x_i$ of the *correct class* y_i, while simultaneously pushing down the log-sum of the scores of *all available classes*. This creates a competitive environment where increasing the score of the correct class implicitly decreases the relative probability of the others.

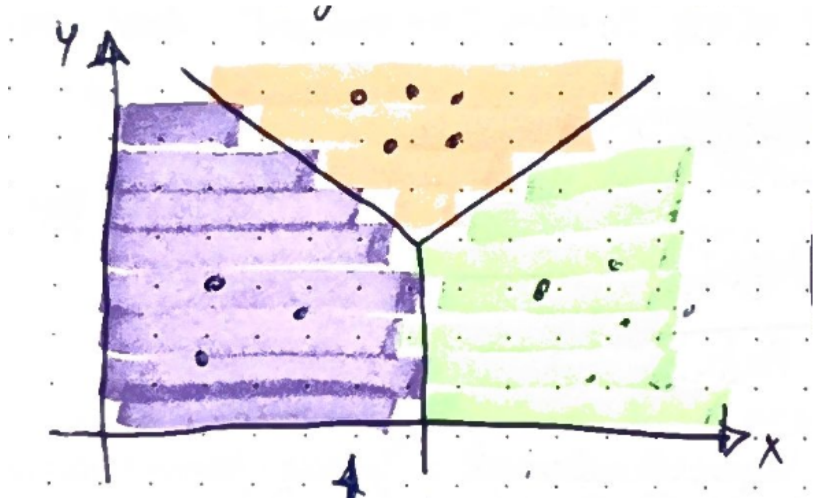


Figure 1.3: Illustration of the Sigmoid function in Logistic Regression.

1.4 Advanced Optimization and Regularization

1.4.1 Regularization

The primary goal of Regularization is to prevent the model from becoming too complex and overfitting the training data (i.e., we use it to combat **High Variance**). We achieve this by adding a penalty term $R(W)$, based only on the model's parameters, directly to the Data Loss.

The new Total Loss becomes:

$$L_{total}(W,X) = \underbrace{L(W,X)}_{\text{Data Loss}} + \underbrace{\lambda R(W)}_{\text{Regularization Term}}$$

Note

When and how is it applied?
Regularization is applied *during training*. By redefining the Total Loss, we also change its gradient. When we apply Gradient Descent, the algorithm must now minimize both the data error AND the complexity of the weights. For example, the gradient of an L2 penalty λW^2 is $2\lambda W$, meaning Gradient Descent will actively shrink the weights at every step (Weight Decay).

The hyperparameter λ acts as the "manipulator" of complexity:

- $\lambda = 0$: We ignore complexity. The model relies entirely on Data Loss (Risk of **Overfitting**).
- $\lambda \to \infty$: The model becomes "lazy". The penalty is so high that Gradient Descent brings all weights to zero (Risk of **Underfitting**).

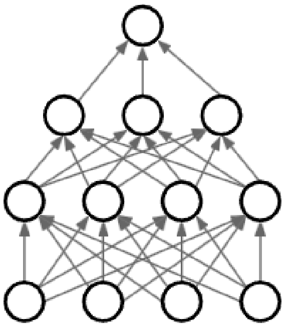
The two most classic mathematical formulations are:

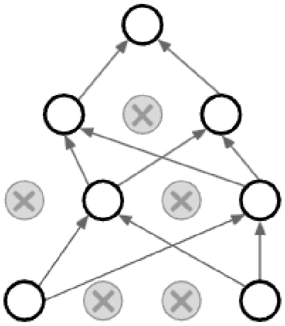
- **L2 Regularization (Ridge):** $R(W) = \sum W^2$. It punishes larger weights more severely, forcing the network to distribute the "weight" across all features rather than relying heavily on just a few. It shrinks weights but rarely brings them exactly to zero.
- **L1 Regularization (Lasso):** $R(W) = \sum |W|$. It punishes weights constantly. This has the mathematical effect

of bringing less important weights exactly to zero, acting as an automatic *Feature Selection* mechanism.

While L1 and L2 regularization act directly on the loss function, modern Deep Learning heavily relies on *architectural* and *stochastic* regularization techniques.

1.4.2 Dropout





First introduced by Srivastava et al. (2014), **Dropout** is a simple yet incredibly effective regularization technique for neural networks.

How it works: During training (in each forward pass), we randomly set a fraction of neurons to zero. The probability of dropping a neuron is a hyperparameter, with $p = 0.5$ being a common default for fully connected layers.

- **Why does it work?**
  1. **Prevents Co-adaptation:** It forces the network to learn a *redundant representation*. If a neuron cannot rely on the presence of specific other neurons, it must learn robust, independent features (e.g., detecting a cat's ear rather than memorizing the whole cat's face).
  2. **Ensemble Interpretation:** Dropout can be seen as training a massive ensemble of sub-networks that share parameters. An FC layer with 4096 units has 2^{4096} possible binary masks (sub-models)!

The Solution: Inverted Dropout

Dropping neurons only during *training* means that at *test time* (when all neurons are active), the total output sum will be **much larger**. For example, if $p = 0.5$ (Dropout Rate), the expected output at test time is twice as large. Instead of modifying the network at test time, we apply a c̈orrectiond̈irectly during training. We define the training activation h as:

$$h_{\text{train}} = \frac{\text{mask} \odot x}{1 - p}$$

Let's break down exactly what this formula does:

- **The Numerator** ($\text{mask} \odot x$): This is the actual dropout operation. The `mask` is a matrix of 0s and 1s (where a 0 appears with probability p). By performing element-wise multiplication ($\odot$), we force a fraction p of the activations in x to become exactly zero, effectively ẗurning offẗhose neurons for the current pass.
- **The Denominator** ($1 - p$) **Pumping part**: This is the k̈eep probabilitys̈caling factor. Since we turned off a fraction p of the neurons, the total sum of the layer's output dropped. By dividing the *surviving* neurons by $(1 - p)$, we artificially inflate their values. This mathematically guarantees that the *expected value* of the layer remains identical to what it will be at test time.

Thanks to this trick, at test time the network requires no scaling at all, running at maximum efficiency:

$$h_{\text{test}} = x$$

Example

Numerical Example of Inverted Dropout ($p = 0.5$)

Given an input $x = [10, 20, 30, 40]$ (Sum $= 100$) and dropout rate $p = 0.5$:

1. Generate Binary Mask: Random mask with 50% probability of being 0.

$$\text{mask} = [1, 0, 1, 0]$$

2. Apply Numerator (Masking): Element-wise multiplication drops the 2nd and 4th neurons (Sum drops to 40).

$$\text{mask} \odot x = [1, 0, 1, 0] \odot [10, 20, 30, 40] = [10, 0, 30, 0]$$

3. Apply Denominator (Scaling): Divide by $(1 - p) = 0.5$ to inflate surviving neurons, restoring signal energy (Sum becomes $80 \approx 100$).

$$h_{\text{train}} = \frac{[10, 0, 30, 0]}{0.5} = [20, 0, 60, 0]$$

1.4.3 The Common Pattern of Modern Regularization

Dropout is just one example of a broader paradigm in deep learning regularization. The general framework is:

1. **Training:** Add some kind of randomness or noise.
2. **Testing:** Average out (or marginalize) the randomness to produce a deterministic prediction.

Other notable examples following this pattern include:

- **Batch Normalization:** Injects randomness during training because a sample's normalization depends on the random composition of its minibatch. At test time, global running statistics are used instead.
- **Data Augmentation:** Randomly crops, flips, or color-jitters training images.
- **Mixup (Zhang et al., 2018):** Randomly blends the pixels of pairs of training images (e.g., 40% cat, 60% dog) and correspondingly blends their target labels.
- **Cutout:** Randomly masks out square regions of input images during training.

1.4.4 Stochastic Gradient Descent (SGD)

From a theoretical standpoint, our objective is to minimize the total loss over the entire dataset, which includes both the data loss and the regularization term:

$$L(W) = \frac{1}{N} \sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

To update the weights, we need the exact gradient of this total loss:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Computing this true gradient (the **Full Batch** gradient) requires evaluating the sum over all N samples in the dataset at every single step. When N is in the millions, this becomes computationally unfeasible.

Key Word

Stochastic Gradient Descent (SGD): Instead of the full sum, SGD approximates the true gradient by computing the sum over a small **minibatch** of examples (commonly 32, 64, or 128). This introduces noise but allows for much faster and frequent parameter updates.

1.5 Backpropagation and Neural Networks

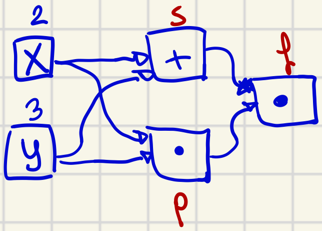
1.5.1 Backpropagation

To update weights, we compute gradients via **Backpropagation**, which applies the chain rule from the output back to the inputs (backward pass).

Note

Backprop with Matrices: When inputs and weights are vectors or matrices, local gradients become *Jacobian matrices*. However, Jacobians are often sparse. Instead of explicitly forming them, we use implicit multiplication. A very useful trick to remember: the downstream gradient dL/dx **always has the same shape** as x!

Here are some solved exercises demonstrating the Forward and Backward pass step-by-step:



FORWARD PASS

$s = x+y = 2+3 = 5$

$p = x\cdot y = 2\cdot 3 = 6$

$f = s\cdot p = 30$

Reverse Pass

- $f = s\cdot p$ $\quad \frac{\partial f}{\partial s} = p \quad \frac{\partial f}{\partial p} = s$
- $s = x+y$ $\quad \frac{\partial s}{\partial x} = 1+0=1 \quad \frac{\partial s}{\partial y} = 0+1=1$
- $p = x\cdot y$ $\quad \frac{\partial p}{\partial x} = y \quad \frac{\partial p}{\partial y} = x$

Applico la chain Rule

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial s}\cdot\frac{\partial s}{\partial x} + \frac{\partial f}{\partial p}\cdot\frac{\partial p}{\partial x} = p\cdot 1 + s\cdot y = 6\cdot 1 + 5\cdot 3 = 21$$

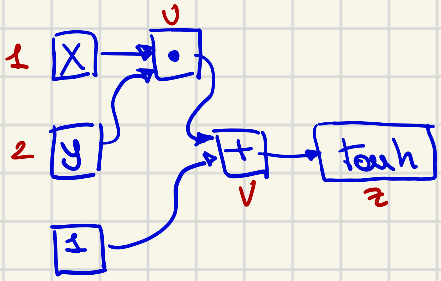
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial s}\cdot\frac{\partial s}{\partial y} + \frac{\partial f}{\partial p}\cdot\frac{\partial p}{\partial y} = p\cdot 1 + s\cdot x = 6\cdot 1 + 5\cdot 2 = 16$$

Cosa capisco da questo esercizio? Capisco quanto un peso x e y impattano rispetto alla funzione di Loss f. Vedo che X ha un impatto maggiore.

Esercizio 3

Consider a 2-Layer function: $z = \tanh(xy+1)$ $\quad x=1$ e $y=2$

- Build a computation graph:
- forward pass: compute z.
- Reverse pass: compute gradient $\frac{\partial z}{\partial x}, \frac{\partial z}{\partial y}$.



FORWARD:

- $u = x\cdot y = 2$
- $v = u+1 = 3$
- $z = \tanh(v) = \tanh(3) \cong 0{,}995$

Reverse Pass

- $z = \tanh(v) \rightsquigarrow \frac{\partial z}{\partial v} = \text{sech}^2(v) = 1 - \tan^2(v) \cong 1-(0.995)^2 \cong 0.01$
- $v = u+1 = \rightsquigarrow \frac{\partial v}{\partial u} = 1$ $\quad$ • $u = x\cdot y \rightsquigarrow \frac{\partial u}{\partial x} = y = 2$

$\frac{\partial u}{\partial y} = x = 1$

- Chain Rule

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial v}\cdot\frac{\partial v}{\partial u}\cdot\frac{\partial u}{\partial x} = 0{,}01\cdot 1\cdot 2 \cong 0{,}02$$

$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial v}\cdot\frac{\partial v}{\partial u}\cdot\frac{\partial u}{\partial y} = 0{,}01\cdot 1\cdot 1 \cong 0{,}01$$

Esercizio 4

given: $g(w) = (w-3)^2$ $\quad w_0 = 0$ $\quad \alpha = 0{,}1$

$$\boxed{g'(w) = 2(w-3)}$$

$$w^k = w^{k-1} - \alpha \nabla g'(w^{k-1})$$

① WRITE the gradient descent Rule

② Compute the first 5 iter... $w_1 \dots w_5$

③ sketch the cost function values.

$$w_{k+1} = w_k - \alpha g'(w_k) =$$
$$w_{k+1} = w_k - 0{,}2(w_k - 3) =$$
$$w_{k+1} = w_k - 0{,}2\,w_k + 0{,}6$$
$$w_{k+1} = (1-0{,}2)\,w_k + 0{,}6$$
$$w_{k+1} = 0{,}8\,w_k + 0{,}6$$

Notiamo come solo lo 98% del vecchio valore è mantenuto.

$w_0 = 0$

$w_1 = 0{,}8 \cdot 0 + 0{,}6 = 0{,}6$

$w_2 = 0{,}8 \cdot 0{,}6 + 0{,}6 = 1{,}08$

$w_3 = 0{,}8 \cdot 1{,}08 + 0{,}6 = 1{,}464$

$w_4 = 0{,}8 \cdot 1{,}464 + 0{,}6 = 1{,}7712$

$w_5 = 0{,}8 \cdot 1{,}7712 + 0{,}6 = 2{,}01696$

Ad ogni passo l'iterazione ci spinge un po' più vicino al minimo vero $w^* = 3$. La velocità con cui ci avviciniamo a 3 diminuisce gradualmente.

ESERCIZIO 5

$$g(w_1, w_2) = w_1^2 + 2w_2^2$$

① Compute the gradient $\nabla g(w)$

② Apply GD from initial point $(w_1, w_2) = (2, 1)$ with learning rate $\alpha = 0{,}2$

③ Compute the first 3 update explicitly.

for computing the gradient $\nabla g(w_1, w_2)$ we have to apply the first derivate to the function $g(w_1, w_2) \rightsquigarrow g'(w_1, w_2) = 2w_1 + 4w_2$

$$\nabla g(w_1, w_2) = \begin{bmatrix} 2w_1 \\ 4w_2 \end{bmatrix}$$

Il gradiente è la Derivata della funzione.

Rule for each weight:

$\boxed{w_1}$

$$w_1^{k+1} = w_1^k - \alpha \nabla g(w_1) =$$
$$w_1^{k+1} = w_1^k - 0{,}2\,(2w_1^k) =$$
$$w_1^{k+1} = w_1^k - 0{,}4\,w_1^k =$$
$$\boxed{w_1^{k+1} = 0{,}6\,w_1^k}$$

$\boxed{w_2}$

$$w_2^{k+1} = w_2^k - \alpha \nabla g(w_2^k) =$$
$$w_2^{k+1} = w_2^k - 0{,}2\,(4w_2^k)$$
$$w_2^{k+1} = w_2^k - 0{,}8\,w_2^k =$$
$$\boxed{w_2^{k+1} = 0{,}2\,w_2^k}$$

Questo è il modo in cui cambieremo i Pesi.

Let's compute the iteration starting with $(2, 1)$

$K=1 \rightsquigarrow (w_1, w_2) = (0.6 \cdot 2,\ 0.2 \cdot 1) = (1.2,\ 0.2)$

$K=2 \rightsquigarrow (1.2,\ 0.2) = (0{,}72,\ 0{,}04)$

$K=3 \rightsquigarrow (0{,}74,\ 0.04) = (0{,}432,\ 0{,}008)$

Obiettivo: Vogliamo capire come la funzione di LOSS guida l'aggiornamento dei Pesi per far sì che la nostra Rete produca output sempre più vicini a quelli desiderati.

EX.

Immagina di avere un modello lineare usiamo ad Esempio $(x, y) = (2, 5)$

$$\hat{y} = w \cdot x + b$$

Peso

Bias

input

output Predetto.

PASSO 1: Inizializzo i PESI

Pesi iniziali: $w = 0{,}5 \quad b = 0{,}0$

! Noi non sappiamo nulla quindi impostiamo valori casuali.

PASSO 2: FORWARD PASS (PREDIZIONE)

$\hat{y} = w \cdot x + b = 0.5 \; 2 + 0 = 1$ ← PREDIZIONE

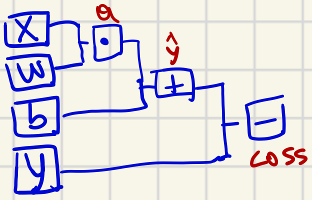
PASSO 3: calcolo della LOSS (ERRORE). usiamo MSE

$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{(1-5)^2}{2} = \frac{16}{2} = 8$ ← siamo molto lontani dal TARGET.

PASSO 4: Backpropagation (calcolo del Gradiente): come cambiamo i Pesi ??

❗ Deriviamo la Loss Rispetto a $\hat{y}$

GRAFO COMPUT:



$$\begin{cases} a = w \cdot x \\ \hat{y} = a + b \\ L = \hat{y} - y \end{cases}$$

BACKWARD PASS

- $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$
- $\frac{\partial \hat{y}}{\partial a} = 1$
- $\frac{\partial a}{\partial w} = x$
- $\frac{\partial \hat{y}}{\partial b} = 1$

$\boxed{w} \rightsquigarrow \frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a} \cdot \frac{\partial a}{\partial w} = (\hat{y} - y) \cdot x = (-4) \cdot 2 = \boxed{-8}$

$\boxed{b} \rightsquigarrow \frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b} = (\hat{y} - y) \cdot 1 = \boxed{-4}$

PASSO 5: GD update dei Pesi. Regola di aggiornamento: $w := w - \alpha \frac{\partial L}{\partial w}$

$\alpha = 0{,}1$

$b := b - \alpha \frac{\partial L}{\partial b}$

$w^1 = 0.5 - 0.1 \cdot (-8) = 1{,}3$ NUOVI PESI $k=1$

$b^1 = 0 - 0.1 \cdot (-4) = 0{,}4$

PREDICT

$\hat{y} = w^1 x + b^1 = 1{,}3 \cdot 2 + 0{,}4 = 3$ Nota come la predizione diventa più precisa e vicina al 5 del DATASET.

Ovviamente questo Processo viene eseguito su ogni Riga del Dataset. Non sulla stessa.

1.5.2 Deeper Neural Networks

By stacking multiple layers and applying non-linear activation functions, we create multi-layer Neural Networks (e.g., $f = W_2 \max(0, W_1 x)$).

1.6 The Role of Activation Functions

An **activation function** is a mathematical "gate" applied to the output of a layer ($z = Wx + b$) before passing it to the next layer. Its primary role is to introduce **non-linearity** into the network.

> **Note**
>
> **Why do we need Non-Linearity?** If we only stacked linear layers without activation functions, the entire deep network would mathematically collapse into a single linear transformation. For example, a 2-layer linear network $f = W_2(W_1 x)$ can be rewritten as $f = W'x$ where $W' = W_2 W_1$. Non-linear activations allow the network to learn and approximate highly complex, non-linear boundaries.

Some of the most common activation functions include:

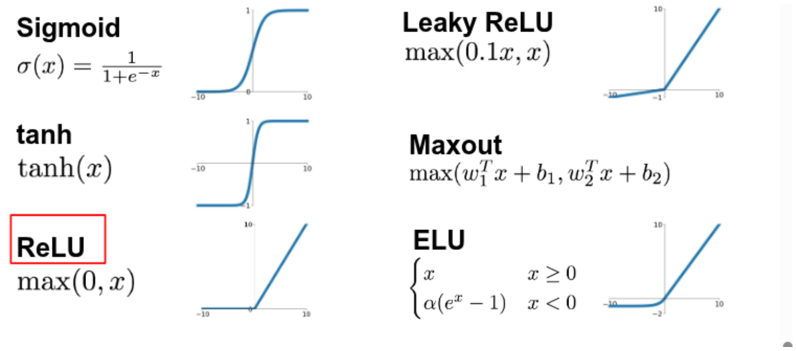


Figure 1.4: Illustration Activation Function.

1.6.1 1. ReLU (Rectified Linear Unit)

Formula: $f(x) = \max(0, x)$

ReLU is the most widely used default activation function in Deep Learning.

- **Pros:** It does not saturate in the positive region. It is highly computationally efficient (just a thresholding operation) and converges much faster than Sigmoid/Tanh in practice (e.g., roughly 6× faster).
- **Cons:** Its outputs are not zero-centered. Most importantly, it suffers from the **"Dead ReLU"** problem: if $x < 0$, the gradient is exactly 0. If a neuron is pushed into the negative regime by a large weight update, its weights will never update again.

1.6.2 2. Leaky ReLU

Formula: $f(x) = \max(\alpha x, x)$ (where typically $\alpha = 0.01$)

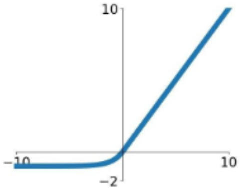
Designed to solve the "dead neuron" problem of standard ReLU.

- **Pros:** Retains all the benefits of ReLU (no positive saturation, computationally efficient, fast convergence), but the small non-zero slope for negative inputs ensures the neurons will not "die".

1.6.3 3. ELU (Exponential Linear Units)

Formula: $f(x) = x$ if $x > 0$, else $\alpha(e^x - 1)$

- **Pros:** Has all the benefits of ReLU, but produces outputs that are closer to zero-mean. Furthermore, its negative saturation regime adds some robustness to noise compared to the unbounded negative regime of Leaky ReLU.
- **Cons:** Computation is more expensive because it requires evaluating the exponential function e^x.



$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

Figure 1.5: Illustration of a ELU

1.6.4 4. GELU (Gaussian Error Linear Unit)

Formula: $f(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is the cumulative distribution function (CDF) of the standard normal distribution.

- **Pros:** Exhibits very nice behavior around 0. Its mathematical smoothness facilitates training in practice. It is widely used in state-of-the-art models like **GPT-3, BERT**, and most other Transformers.
- **Cons:** Higher computational cost than ReLU. Note that for large negative values, the gradient still tends to 0.

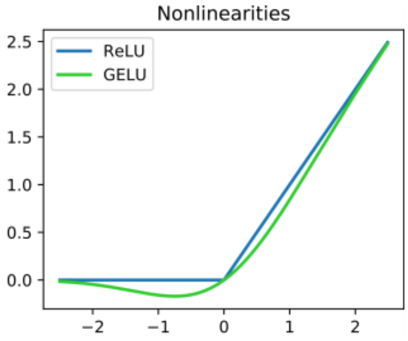


Figure 1.6: Illustration of a GELU

Note

TL;DR Practical Guidelines for Activation Functions:

- Use **ReLU** as your standard default, but be careful with your learning rates to avoid dead neurons.
- Try out **Leaky ReLU**, **PReLU** (Parametric ReLU), or **GELU** to squeeze out some marginal performance gains.
- Generally avoid **Sigmoid/Tanh** for hidden layers unless specifically required by the architecture (e.g., LSTM gates).

1.7 Recap on Convolutional Neural Networks (CNNs)

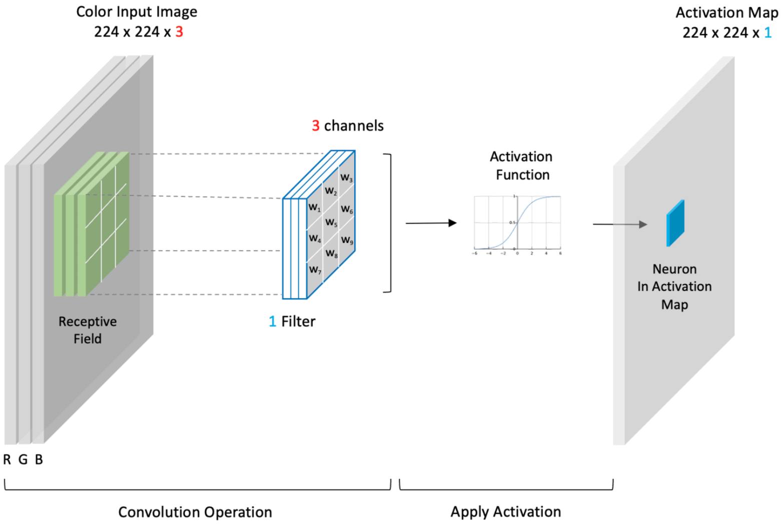

Color Input Image
224 x 224 x 3
Activation Map
224 x 224 x 1
3 channels
Activation
Function
w_1
w_2
w_3
w_4
w_5
w_6
w_7
w_8
w_9
Neuron
In Activation
Map
Receptive
Field
1 Filter
R G B
Convolution Operation
Apply Activation

Figure 1.7: Illustration of a single convolutional layer.

1.7.1 Motivation: The Problem with Fully Connected Layers

In a standard Fully Connected (Dense) layer, every input neuron is connected to every output neuron. If we process a high-resolution image for example, a 1000×1000 RGB image (3×10^6 inputs) even a modest layer with 10^3 hidden units would require over 3 billion parameters.

This huge parameter count leads to two main issues:

- **Computational Inefficiency & Overfitting:** Storing and updating billions of weights requires massive memory and easily leads to severe overfitting.
- **Loss of Spatial Structure:** Flattening a 2D/3D image into a 1D vector completely ignores the grid-like topology of pixels.

1.7.2 Exploiting Image Structure: Locality and Stationarity

To overcome these limitations, Convolutional Neural Networks (**CNN**) build upon two fundamental spatial properties of images:

1. **Local Spatial Correlation:** Nearby pixels are strongly related, whereas distant pixels are less relevant. Instead of connecting a unit to the whole image, we can connect it only to a small local patch (a *receptive field*).
2. **Stationarity (Translation Invariance):** Features are location-independent. An edge, texture, or pattern detected in the top-left corner is equally meaningful in the bottom-right corner. Therefore, we can share the same set of weights across all locations.

1.7.3 The Core Building Block: A Simple Convolutional Layer

A Convolutional Layer replaces the dense matrix multiplication with a local, sliding operation.

- **The Kernel (Filter):** A small matrix of learnable weights (e.g., 3×3 or 5×5). It acts as a pattern detector.
- **The Sliding Window:** The kernel slides across the input image step-by-step. At each position, it computes an element-wise multiplication with the underlying image patch and sums the results up.
- **The Feature Map:** The output grid produced by sliding the kernel, Figure1.8. High values in the feature map indicate a strong match between the kernel's pattern and that specific image region.

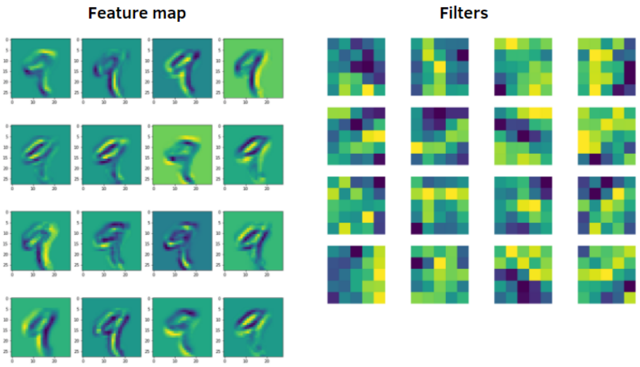


Figure 1.8: Illustration of multiple features map obtained by different filters

1.7.4 Key Hyperparameters: Controlling the Convolution

The behavior and output size of a convolutional layer depend on three primary parameters:

- **Stride (S):** The step size with which the kernel moves across the image. A stride of 1 moves pixel-by-pixel, while $S > 1$ downsamples the output.
- **Padding (P):** Adding zero-valued borders around the input image to preserve spatial dimensions and process edge pixels effectively.
- **Input/Output Channels (C):** Real images have depth (e.g., $C_{in} = 3$ for RGB). A kernel must match the depth of the input, resulting in a 3D tensor of shape $(C_{in} \times K_h \times K_w)$. Using C_{out} distinct kernels produces C_{out} output feature maps.

1.7.5 Formal Mathematical Definition

Mathematically, the operation performed on a 2D input matrix I by a 2D kernel g to yield a feature map f at position $[m,n]$ is defined as:

$$f[m,n] = (I * g)[m,n] = \sum_k \sum_l I[m-k, n-l]\, g[k,l] \tag{1.1}$$

where k and l iterate over the spatial dimensions of the kernel.

Note on Deep Learning vs. Signal Processing:

In classic signal processing, the index subtraction $I[m-k, n-l]$ implies flipping the filter g by 180°. However, deep learning frameworks (like PyTorch and TensorFlow) compute **Cross-Correlation** instead:

$$f[m,n] = (I \star g)[m,n] = \sum_k \sum_l I[m+k, n+l]\, g[k,l] \tag{1.2}$$

Skipping the flipping step saves computation. Since kernel weights $g[k,l]$ are learned from scratch via backpropagation, the network simply learns the pre-flipped weights directly, making the distinction mathematically equivalent for optimization.

1.7.6 Intuitive Examples: Classic Image Processing Filters

Before moving to 3D volumes, let us observe how simple 2D matrices (kernels) process local pixel neighborhoods to extract features or alter images.

Example

1. Smoothing Filter (Noise Reduction):

Replacing each pixel with a weighted combination of its local neighborhood dampens noise and sharp anomalies:

$$\begin{array}{|c|c|c|} \hline 2 & 3 & 3 \\ \hline 3 & \mathbf{20} & 2 \\ \hline 3 & 2 & 3 \\ \hline \end{array} \xrightarrow{\text{Smoothing}} \begin{array}{|c|c|c|} \hline 2 & 3 & 3 \\ \hline 3 & \mathbf{3} & 2 \\ \hline 3 & 2 & 3 \\ \hline \end{array}$$

The central outlier value (20) is effectively smoothed down to 3.

Example

2. Vertical Edge Detection Filter:

Filters with opposing positive and negative weights capture spatial variations in intensity. For example, a vertical edge detector is defined as:

$$g_{\text{vertical}} = \begin{array}{|c|c|c|} \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline \end{array}$$

Note: Transposing g_{vertical} yields a horizontal edge detector $g_{\text{horizontal}} = g_{\text{vertical}}^T$.

Example

3. Step-by-Step 2D Convolution Example (Stride 1, Pad 1):

Consider convolving a 3×3 image I with a 3×3 vertical edge filter g. Applying **Padding** $P = 1$ means we add a border of zeros around I, resulting in a 5×5 padded image I_{padded}:

$$I_{\text{padded}} = \begin{array}{|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 8 & 5 & 2 & 0 \\ \hline 0 & 7 & 5 & 3 & 0 \\ \hline 0 & 9 & 4 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 \\ \hline \end{array}$$

Applying the convolution with **Stride** $S = 1$ over this padded image gives the output feature map f:

$$\begin{array}{c} g[k,l] \\ \begin{array}{|c|c|c|} \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline \end{array} \end{array} \otimes \begin{array}{c} I_{\text{padded}} \\ \text{(conceptually)} \end{array} = \begin{array}{c} f[m,n] \\ \begin{array}{|c|c|c|} \hline -10 & 10 & 10 \\ \hline -14 & 18 & 14 \\ \hline -9 & 12 & 9 \\ \hline \end{array} \end{array}$$

Calculation Detail (Accounting for 180° Kernel Flip):

According to $f[m,n] = \sum I[m-k, n-l]g[k,l]$, the kernel g is flipped by 180° prior to element-wise multiplication:

$$g_{\text{flipped}} = \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array}$$

- **Center Value** $f[1,1] = 18$**:** Flipped kernel overlaps fully with the original 3×3 image pixels in I_{padded}:
$$(1 \cdot 8 - 1 \cdot 2) + (1 \cdot 7 - 1 \cdot 3) + (1 \cdot 9 - 1 \cdot 1) = 6 + 4 + 8 = 18$$
- **Top-Left Value** $f[0,0] = -10$**:** Flipped kernel overlaps with the top-left 3×3 corner of I_{padded}:
$$(1 \cdot 0 - 1 \cdot 5) + (1 \cdot 0 - 1 \cdot 5) = -10$$

1.7.7 Output Shape and Parameter Counting for 3D Volumes

In practical deep learning architectures, inputs are 3D tensors $(C_{\text{in}} \times H \times W)$ and layers use multiple filters to extract distinct visual features.

Spatial Output Size Formula

Given an input of spatial size W, kernel size K, padding P, and stride S, the output dimension W_{out} is given by:

$$W_{\text{out}} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \tag{1.3}$$

Example

Example: Output Volume Size & Parameters Calculation

If the input volume is a $32 \times 32 \times 3$ image, and we apply **10 filters of size** 5×5 (with Stride $S = 1$, Padding $P = 2$):

- **Output spatial size:** $(32 + 2 \cdot 2 - 5)/1 + 1 = 32$ spatially (32×32).
- **Output volume size:** $32 \times 32 \times 10$.
- **Number of parameters:** Each filter has $5 \times 5 \times 3 + 1\,(\text{bias}) = 76$ parameters. For 10 filters: $76 \times 10 = \mathbf{760}$ parameters.

1.7.8 The Pooling Layer

A Convolutional Neural Network usually interleaves Convolutional layers with Pooling layers.

Purpose of Pooling:

The primary goal of a pooling layer is to perform **spatial downsampling** (subsampling). By doing so, it:

- **Reduces the spatial dimensions** (width and height) of the representation, making it smaller and more computationally manageable.
- **Reduces the number of parameters** in the subsequent fully connected layers, thus helping to control overfitting.
- **Provides Translation Invariance:** Small shifts or distortions in the input image will not drastically change the pooled output, making the network more robust to exact feature locations.

How it Operates:

Unlike the convolution layer, a pooling layer **operates independently on every depth slice** (activation map) of the input. It does not mix channels. If the input has C channels, the output will also have C channels.

Max Pooling

The most common implementation is **Max Pooling**. Instead of computing a dot product like a filter, it simply slides a window across the image and takes the **maximum value** within that window.

Example

Max Pooling Example (Stride 2, Filter 2×2):

Consider a single depth slice (activation map) of size 4×4. If we apply a Max Pooling layer with a 2×2 filter and a stride of 2:

$$\begin{array}{|c|c|c|c|}\hline 1 & 1 & 2 & 4 \\ \hline 5 & 6 & 7 & 8 \\ \hline 3 & 2 & 1 & 0 \\ \hline 1 & 2 & 3 & 4 \\ \hline \end{array} \xrightarrow{\text{Max Pool } 2\times 2, S=2} \begin{array}{|c|c|}\hline 6 & 8 \\ \hline 3 & 4 \\ \hline \end{array}$$

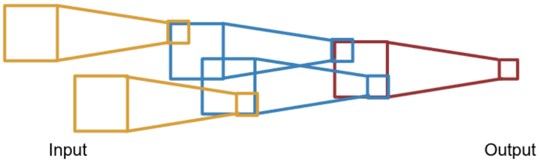
The spatial dimensions are exactly halved. The operation discards 75% of the activations, keeping only the most prominent features (highest numbers) in each local 2×2 region.

1.7.9 Visualizing ConvNets

Key Insight

Understanding what deep CNNs learn requires visualization techniques (e.g., Deconvolutional Networks by Zeiler and Fergus). By projecting high-level feature maps back to the pixel space:

- **Layer 1:** Learns low-level features like edges and colors.
- **Layer 2:** Learns simple textures and geometric patterns.
- **Layer 3 & Higher:** Learns complex, object-specific parts (e.g., dog faces, car wheels) and eventually full objects.



Bibliography

[1] Fabio Galasso. *Recap on Image Classification and Neural Networks*. Advanced Machine Learning Course Slides, Sapienza Università di Roma. Sept. 2025.